DATASTOCK

Práctica Integradora

ENTREGA 2 · UT2

Acceso a datos

2.º DAM · Curso 2026–2027

IES ISIDRA DE GUZMÁN · ALCALÁ DE HENARES

Contenido

1. Contexto y continuidad de la práctica 3
2. Punto de partida y materiales 3
3. Modelo relacional obligatorio 3
3.1. Validaciones de datos y restricciones de integridad en MySQL 4
3.1.1. Restricciones del código de producto 4
3.1.2. Restricciones del nombre del producto 4
3.1.3. Restricciones del precio 5
3.1.4. Restricciones del stock 5
3.1.5. Restricciones de categoría y proveedor: integridad referencial 6
3.1.6. Restricciones al modificar o eliminar registros 7
3.1.7. Respeto de las restricciones del script SQL 7
4. Preparación del entorno y primera conexión 8
5. Interfaz de usuario: ampliación de los menús 8
5.1. Convenciones generales y mensajes de la consola 8
5.2. Menú principal: conservar la UT1 9
5.3. Submenú 7.2: categorías (RF-JDBC-02) 9
5.4. Submenú 7.3: proveedores (RF-JDBC-02) 9
5.5. Submenú 7.4: mantenimiento de productos (RF-JDBC-03) 10
5.6. Submenú 7.5: consultas de productos (RF-JDBC-04) 10
5.7. Submenú 7.6: movimientos de stock (RF-JDBC-05) 11
5.8. Submenú 7.7: historial (RF-JDBC-06) 11
5.9. Opción 7.8: copiar inventario de memoria a MySQL (RF-JDBC-07) 12
5.10. Opción 7.9: demostración de transacciones JDBC (RF-JDBC-08) 12
6. Requisitos funcionales de la UT2 13
RF-JDBC-01. Conectar y comprobar el esquema 13
RF-JDBC-02. Gestionar categorías y proveedores 13
RF-JDBC-03. CRUD de productos 13
RF-JDBC-04. Consultas con ResultSet 14
RF-JDBC-05. Registrar movimientos con procedimiento almacenado 14
RF-JDBC-06. Consultar historial de movimientos 15
RF-JDBC-07. Importación controlada desde la UT1 15
RF-JDBC-08. Transacción JDBC de demostración 15
5.11. Ejemplo completo de recorrido de usuario 16
6.1.1. Situación inicial 16
6.1.2. Paso 1. Importar los productos desde un fichero JSON (UT1) 16
6.1.3. Paso 2. Acceder a las funcionalidades de MySQL (UT2) 17

6.1.4. Paso 3. Dar de alta las categorías necesarias 18
6.1.5. Paso 4. Dar de alta el proveedor necesario 18
6.1.6. Paso 5. Importar los productos almacenados en memoria a MySQL 19
6.1.7. Paso 6. Consultar los productos almacenados en MySQL 20
6.1.8. Paso 7. Cerrar DataStock y volver a ejecutar la aplicación 21
6.1.9. Paso 8. Completar los datos pendientes y repetir la importación 21
6.1.10. Comprobaciones finales del recorrido 22
7. Actividad comparativa obligatoria: gestor embebido e independiente 23
7.1. Instalación y dependencia JDBC SQLite 23
7.2. Base de datos de demostración y tabla 23
7.3. Desarrollo paso a paso de DemoSQLite 23
7.4. Comparación escrita con MySQL 24
7.5. Pruebas y evidencias de la actividad 24
8. Reglas técnicas y de seguridad 24
9. Evaluación y relación con los criterios de evaluación 24
10. Entregables de la segunda fase 25
11. Lista final de comprobación de la entrega UT2 25
11.1. Continuidad del proyecto y funcionalidades de la UT1 25
11.2. Preparación de MySQL y configuración JDBC 26
11.3. Menús, navegación y presentación de resultados 27
11.4. Gestión de categorías 27
11.5. Gestión de proveedores 28
11.6. Gestión de productos 28
11.7. Consultas y búsquedas de productos 29
11.8. Entradas y salidas de stock 30
11.9. Historial de movimientos 31
11.10. Importación de productos desde memoria a MySQL 31
11.11. Gestión de transacciones JDBC 32
11.12. Actividad comparativa con SQLite 33
11.13. Buenas prácticas de programación y seguridad 33
11.14. Pruebas funcionales y tratamiento de errores 34
11.15. Comprobación de la persistencia de los datos 35
11.16. Documentación, repositorio y entrega 35
11.17. Prueba final de integración 36

1. Contexto y continuidad de la práctica

DataStock es una única aplicación de consola desarrollada en tres fases. En la UT1 has trabajado con ficheros, JSON, XML y un inventario en memoria. En esta segunda entrega incorporarás una base de datos MySQL real mediante JDBC. No debes comenzar otra aplicación ni eliminar las funcionalidades de ficheros: amplía tu proyecto Maven y conserva el trabajo anterior.

El objetivo profesional es que la empresa pueda mantener un catálogo persistente de categorías, proveedores y productos, y registrar entradas y salidas de existencias con garantías de integridad. El inventario en memoria de la UT1 y la base de datos son almacenes distintos: no deben sincronizarse automáticamente sin una operación explícita.

2. Punto de partida y materiales

- Tu repositorio y proyecto Maven de la UT1, con sus clases de modelo, menús, utilidades y funcionalidades ya implementadas.
- Script facilitado DataStock_BD_MySQL.sql. Debes utilizar su esquema datastock y las tablas categoria, proveedor, producto y movimiento_stock, sin sustituirlas por otro modelo.
- MySQL 8.0.16 o posterior, MySQL Workbench o herramienta equivalente, JDK 21, Maven y MySQL Connector/J.
- Plantilla de registro de pruebas de la práctica, que ampliarás con los casos JDBC.

3. Modelo relacional obligatorio

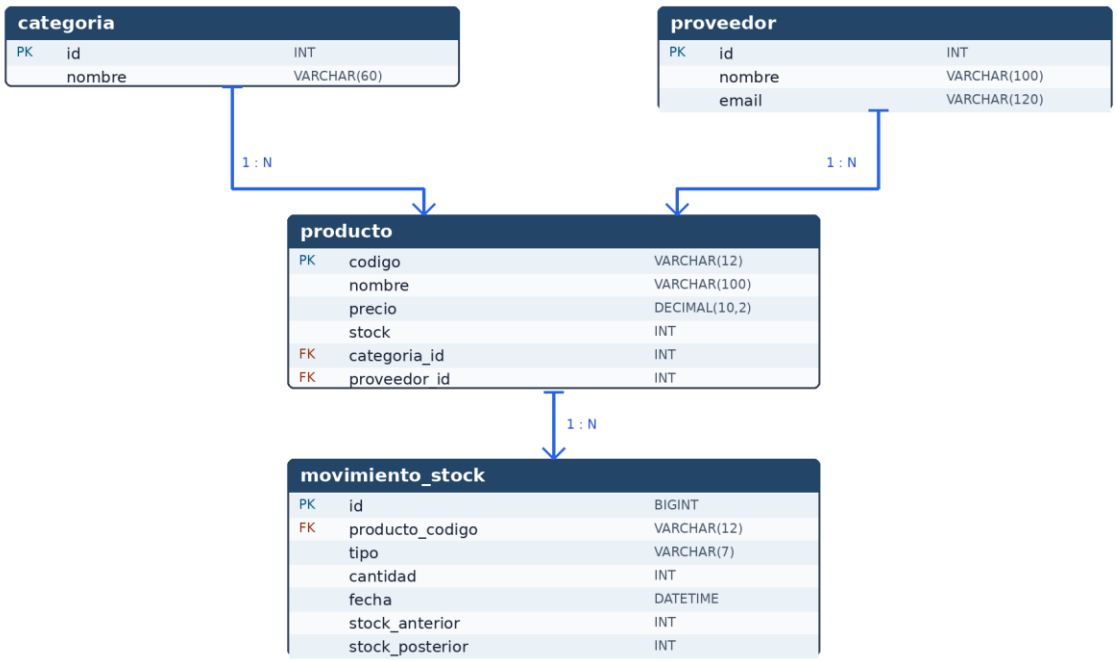


Tabla	Campos principales	Reglas relevantes
categoria	id (PK), nombre	Nombre único; id asignado por el usuario.
proveedor	id (PK), nombre, email	Email opcional y único cuando existe.
producto	codigo (PK), nombre, precio, stock, categoria_id, proveedor_id	Precio > 0; stock >= 0; claves foráneas válidas.
movimiento_stock	id (PK autoincremental), producto_codigo, tipo, cantidad, fecha, stock_anterior, stock_posterior	ENTRADA/SALIDA, cantidad > 0; historial de movimientos.

3.1. Validaciones de datos y restricciones de integridad en MySQL

En la UT1, los productos importados desde ficheros JSON o XML se almacenaban en una colección de objetos Java. **Antes de incorporarlos al inventario, la aplicación debía comprobar que sus datos cumplían las reglas de validación establecidas**.

En esta segunda fase, los productos también podrán almacenarse de forma permanente en MySQL. Sin embargo, **no será suficiente con comprobar que los datos individuales son correctos**. **Será necesario verificar que cumplen las restricciones definidas en la base de datos y que las relaciones entre las distintas tablas son válidas.**

El script SQL proporcionado incluye restricciones destinadas a evitar que se almacene información incorrecta o inconsistente. La aplicación Java deberá respetarlas y gestionar adecuadamente los errores que puedan producirse.

3.1.1. Restricciones del código de producto

Cada producto se identifica mediante un código único, que constituye su clave primaria en la tabla producto.

El código deberá cumplir las siguientes condiciones:

1. **Longitud:** tendrá entre 1 y 12 caracteres.
2. **Caracteres permitidos:** únicamente podrá contener letras mayúsculas (A-Z), números (0-9) y guiones (-).
3. **Unicidad:** no podrán existir dos productos con el mismo código en MySQL.
4. **Normalización:** si el usuario introduce letras minúsculas, la aplicación podrá convertirlas a mayúsculas antes de realizar la validación y el almacenamiento.

Estas condiciones se reflejan en la siguiente restricción del script SQL:

```
CHECK (codigo REGEXP '^[A-Z0-9-]{1,12}$')
```

Por ejemplo:

Código introducido	Resultado	Explicación
TEC-001	Válido	Cumple las condiciones establecidas.
MON123	Válido	Contiene letras mayúsculas y números.
tec-001	Válido tras normalizar	Puede convertirse a TEC-001.
TEC_001	No válido	El guion bajo no está permitido.
PRODUCTO-12345	No válido	Supera la longitud máxima.

Comportamiento esperado: antes de insertar un producto, DataStock comprobará su código. Si no cumple las condiciones, mostrará un mensaje de error en rojo y no realizará la inserción.

Si el código ya existe en MySQL, se informará de que el producto está duplicado y no se sobrescribirá automáticamente.

3.1.2. Restricciones del nombre del producto

El nombre permite identificar el artículo de manera comprensible para el usuario.

Deberá cumplir las siguientes condiciones:

1. No podrá ser null ni estar vacío.
2. No podrá contener únicamente espacios.

3. Tendrá una longitud mínima de 3 caracteres y máxima de 100.
4. Se eliminarán los espacios innecesarios al principio y al final antes de validar y almacenar el dato.

Por ejemplo, el nombre Teclado mecánico será válido, mientras que TV será rechazado por no alcanzar la longitud mínima.

Comportamiento esperado: si el nombre no cumple las condiciones, se mostrará un mensaje explicativo en rojo y se solicitará un valor correcto.

3.1.3. Restricciones del precio

El precio representa el importe de venta de un producto y se almacena en MySQL mediante el tipo:

DECIMAL(10,2)

Este tipo permite almacenar números decimales exactos, con un máximo de diez cifras en total, de las cuales dos corresponden a la parte decimal.

Para introducir y almacenar el precio se deberán respetar las siguientes reglas:

1. El precio será obligatorio.
2. Deberá ser un número mayor que cero.
3. Podrá contener un máximo de dos cifras decimales.
4. Deberá encontrarse dentro del rango admitido por DECIMAL(10,2).
5. **En Java se utilizará la clase BigDecimal**, adecuada para representar importes monetarios sin los problemas de precisión habituales de float y double.

Por ejemplo:

Precio	Resultado
49.90	Válido
125	Válido
0	No válido
-15.50	No válido
12.345	No válido: supera los dos decimales

Al insertar o modificar el precio mediante JDBC, se utilizará el método PreparedStatement.setBigDecimal().

Comportamiento esperado: DataStock comprobará que el precio es correcto antes de enviarlo a MySQL. Si no lo es, mostrará un mensaje de error en rojo.

3.1.4. Restricciones del stock

El campo stock representa el número de unidades disponibles de un producto.

Deberá cumplir las siguientes condiciones:

1. Será un número entero.
2. No podrá ser negativo.
3. Se admitirá el valor cero, que representa un producto sin existencias.

Por ejemplo, un stock de 15 será válido, al igual que un stock de 0. Sin embargo, un valor de -3 deberá rechazarse.

Importante: durante el alta inicial de un producto se podrá indicar su stock. Sin embargo, una vez almacenado en MySQL, las posteriores entradas y salidas de unidades deberán realizarse mediante la funcionalidad de movimientos de stock.

No se permitirá modificar directamente el stock desde la opción de modificación de productos.

De esta manera, cada cambio posterior quedará registrado en la tabla movimiento_stock, permitiendo conocer el historial de existencias.

3.1.5. Restricciones de categoría y proveedor: integridad referencial

Cada producto debe pertenecer a una categoría y estar asociado a un proveedor.

En la tabla producto existen dos campos que representan estas relaciones:

- categoria_id: identificador de la categoría a la que pertenece el producto.
- proveedor_id: identificador del proveedor asociado.

Estos campos son **claves foráneas**, lo que significa que sus valores deben corresponder a registros existentes en las tablas categoria y proveedor, respectivamente.

Por tanto, antes de insertar un producto en MySQL, la aplicación deberá comprobar:

1. Que el identificador de categoría es un número entero positivo.
2. Que existe una categoría con ese identificador en la tabla categoria.
3. Que el identificador de proveedor es un número entero positivo.
4. Que existe un proveedor con ese identificador en la tabla proveedor.

Ejemplo práctico

Supongamos que se ha importado desde un fichero JSON el siguiente producto:

```
{
  "codigo": "TEC-001",
  "nombre": "Teclado mecánico",
  "precio": 49.90,
  "stock": 10,
  "categoriaId": 1,
  "proveedorId": 2
}
```

El producto cumple las validaciones individuales: su código, nombre, precio y stock son correctos.

Sin embargo, antes de guardarlo en MySQL, será necesario comprobar que existen la categoría 1 y el proveedor 2.

Imaginemos que la base de datos contiene las siguientes categorías:

ID	Nombre
1	Periféricos
2	Monitores

Y los siguientes proveedores:

ID	Nombre
1	Distribuciones Informáticas
3	Componentes del Centro

En este caso, la categoría 1 existe, pero el proveedor 2 no.

Por tanto, **el producto no podrá insertarse en MySQL**, aunque sus restantes datos sean correctos.

La aplicación deberá mostrar un mensaje similar al siguiente, utilizando el color rojo para identificar el error:

```
IMPORTAR PRODUCTOS DE MEMORIA A MYSQL
----------------------------------------

Producto: TEC-001 - Teclado mecánico

[ERROR] No se puede insertar el producto.
        El proveedor con identificador 2
        no existe en la base de datos.

Producto rechazado.
```

¿Cómo se resuelve esta situación?

El usuario deberá acceder al menú de gestión de proveedores y dar de alta el proveedor correspondiente, siempre que disponga de sus datos.

Después podrá repetir la importación del producto.

La aplicación no deberá crear automáticamente categorías o proveedores inexistentes, ya que el fichero JSON únicamente contiene sus identificadores y no necesariamente dispone de toda la información requerida para darlos de alta.

3.1.6. Restricciones al modificar o eliminar registros

Las restricciones de integridad también deberán respetarse al modificar o eliminar información.

Por ejemplo:

1. **Modificar un producto:** si se cambia su categoría o proveedor, el nuevo identificador deberá existir en la tabla correspondiente.
2. **Eliminar una categoría:** si tiene productos asociados, MySQL impedirá su eliminación mientras se mantenga esa relación.
3. **Eliminar un proveedor:** tampoco podrá eliminarse si existen productos que lo referencian.
4. **Eliminar un producto:** si tiene movimientos de stock registrados, MySQL impedirá su eliminación para preservar el historial.

En estos casos, la aplicación deberá capturar la excepción JDBC, mostrar un mensaje comprensible en rojo y permitir que el usuario continúe trabajando.

No se deberán eliminar automáticamente los registros relacionados para conseguir que una operación de borrado tenga éxito.

3.1.7. Respeto de las restricciones del script SQL

El fichero DataStock_BD_MySQL.sql proporcionado contiene las restricciones necesarias para garantizar la integridad de los datos.

Entre ellas se encuentran:

- PRIMARY KEY: impide que existan registros con claves primarias duplicadas.
- UNIQUE: evita valores repetidos en los campos donde se exige unicidad.
- NOT NULL: impide almacenar valores nulos en determinados campos.
- CHECK: comprueba condiciones como que el precio sea positivo o el stock no sea negativo.
- FOREIGN KEY: garantiza que las relaciones entre las tablas sean válidas.

No está permitido modificar o eliminar estas restricciones para facilitar las operaciones JDBC.

El objetivo de la práctica es que aprendáis a desarrollar una aplicación capaz de trabajar correctamente con una base de datos relacional, respetando sus reglas de integridad.

Por tanto, las validaciones deberán realizarse siempre que sea posible desde Java, antes de ejecutar las sentencias SQL. Además, deberán gestionarse las excepciones que MySQL pueda devolver cuando una operación no pueda completarse.

En resumen: en la UT1 comprobábamos que los productos podían incorporarse correctamente a una colección Java. En la UT2 debemos comprobar, además, que esos productos pueden almacenarse en MySQL sin incumplir las restricciones de sus tablas ni las relaciones existentes entre ellas.

4. Preparación del entorno y primera conexión

1. Importa o ejecuta DataStock_BD_MySQL.sql en MySQL y comprueba que existe la base de datos datastock, sus cuatro tablas y el procedimiento registrar_movimiento.
2. Configura el conector MySQL JDBC en pom.xml. Anota en el README la versión de Connector/J utilizada. Lee los datos de conexión a la base de datos de un fichero properties
3. Crea una clase de conexión JDBC (por ejemplo, config/ConexionBD) que devuelva Connection. Obtén URL, usuario y contraseña de variables de entorno o configuración local excluida de Git; nunca publiques contraseñas.
4. Prueba la conexión mediante un SELECT sencillo y muestra un mensaje comprensible si el servidor está apagado, faltan credenciales o el esquema no existe.
5. Utiliza try-with-resources para cerrar Connection, PreparedStatement, Statement y ResultSet cuando corresponda.

5. Interfaz de usuario: ampliación de los menús

Conserva las opciones 1–6 de la UT1 y añade una nueva opción 7. Las funcionalidades de UT1 continuarán utilizando el inventario en memoria. El menú JDBC trabajará directamente con MySQL.

```
DATASTOCK - GESTIÓN DE INVENTARIO
1. Explorar ficheros y directorios (UT1)
2. Importar productos desde JSON (UT1)
3. Importar productos desde XML (UT1)
4. Consultar productos en memoria (UT1)
5. Exportar productos (UT1)
6. Guardar y recuperar inventario (UT1)
7. Gestionar base de datos MySQL (UT2)
0. Salir


GESTIÓN MYSQL - UT2
1. Comprobar conexión y mostrar información de la BD
2. Gestionar categorías
3. Gestionar proveedores
4. Gestionar productos (CRUD)
5. Consultar y buscar productos
6. Registrar entrada o salida de stock
7. Consultar historial de movimientos
8. Importar productos de memoria a MySQL
9. Demostración de transacción con commit/rollback
0. Volver al menú principal
```

Los submenús de categorías, proveedores y productos deben permitir las operaciones especificadas en los requisitos. Se admite variar la presentación de las opciones, pero no omitir funcionalidades. Una operación fallida debe devolver el control al menú, sin cerrar bruscamente la aplicación.

5.1. Convenciones generales y mensajes de la consola

Todas las opciones deben pedir los datos uno a uno, indicar el campo que se está solicitando y permitir regresar al submenú sin finalizar la aplicación. Antes de borrar se solicitará confirmación S/N. Una opción no válida mostrará un aviso y volverá a solicitarse.

Es obligatorio diferenciar visualmente los resultados:

- mensajes de ERROR en rojo ANSI (por ejemplo, código inexistente, clave duplicada, conexión fallida o stock insuficiente);
- operaciones CORRECTAS en verde (alta, actualización, borrado, importación o movimiento completado).
- Se recomienda amarillo para avisos/confirmaciones y color por defecto para menús.

Utiliza la clase de constantes ANSI del proyecto inicial o amplíala; aplica RESET después de cada mensaje. Si la terminal no soporta ANSI, el texto debe seguir siendo comprensible gracias a prefijos como [ERROR] y [OK]. No colorear contraseñas ni mostrar información sensible.

```
[OK] Producto TEC-001 guardado correctamente.  (VERDE)

[ERROR] No existe el proveedor con ID 8.  (ROJO)

[AVISO] ¿Confirmas la eliminación? (S/N):  (AMARILLO)
```

5.2. Menú principal: conservar la UT1

1 Explorar ficheros | 2 Importar JSON | 3 Importar XML | 4 Consultar memoria | 5 Exportar | 6 Serialización | 7 Gestionar MySQL | 0 Salir

Las opciones 1 a 6 seguirán operando exclusivamente sobre el inventario en memoria de la UT1. La opción 7 trabaja con la base MySQL. Los datos NO se sincronizan automáticamente; únicamente 7.8 copia productos desde memoria a MySQL.

5.3. Submenú 7.2: categorías (RF-JDBC-02)

MENÚ DE CATEGORÍAS

1. Dar de alta una categoría
2. Listar categorías
3. Modificar el nombre de una categoría
4. Eliminar una categoría
0. Volver al menú MySQL

Funcionamiento y reglas de cada operación:

- **Alta de categoría:** Solicitar un identificador entero positivo y un nombre obligatorio de 1 a 60 caracteres, eliminando espacios iniciales y finales. Comprobar que no existe ya el ID ni otra categoría con el mismo nombre. Si la inserción se completa, mostrar [OK] en verde; ante duplicados o datos incorrectos, mostrar [ERROR] en rojo sin cerrar el programa.
- **Listado de categorías:** Consultar MySQL y mostrar identificador y nombre, ordenados por ID ascendente. Si la tabla está vacía, indicar «No hay categorías registradas»; esta situación no es un error.
- **Modificación del nombre:** Solicitar el ID de la categoría, comprobar que existe y mostrar el nombre actual. Pedir el nuevo nombre, aplicar las mismas validaciones que en el alta y solicitar confirmación S/N. El identificador no se modifica. Si el ID no existe o el nombre está repetido, explicar el motivo.
- **Eliminación:** Solicitar el ID, recuperar y mostrar los datos de la categoría y pedir confirmación S/N antes de ejecutar DELETE. Si tiene productos asociados, la clave foránea impedirá borrarla: mostrar [ERROR] en rojo y conservar los registros. Si se responde N, cancelar sin cambios.

5.4. Submenú 7.3: proveedores (RF-JDBC-02)

MENÚ DE PROVEEDORES

1. Dar de alta un proveedor
2. Listar proveedores

3. Modificar un proveedor

4. Eliminar un proveedor

0. Volver al menú MySQL

Funcionamiento y reglas de cada operación:

- **Alta de proveedor:** Solicitar ID entero positivo, nombre obligatorio (máximo 100 caracteres) y correo electrónico opcional (máximo 120 caracteres). Si se introduce un correo, comprobar que tiene un formato básico válido y que no está ya registrado; también se rechazará un ID duplicado. El correo puede almacenarse como NULL cuando no se indique.
- **Listado de proveedores:** Mostrar ID, nombre y correo electrónico, ordenados por ID. Si no hay correo, mostrar «—»; si no existen proveedores, presentar un aviso informativo.
- **Modificación:** Pedir ID y comprobar su existencia; mostrar nombre y correo actuales. Permitir introducir un nombre nuevo y cambiar el correo. Explicar claramente las opciones: pulsar Intro para conservar el dato actual y escribir «-» para dejar el correo sin valor (NULL). Validar longitudes y duplicidad de correo antes de actualizar.
- **Eliminación:** Solicitar ID, mostrar los datos y pedir confirmación S/N. Si existen productos asociados, informar del bloqueo por integridad referencial y no borrar ni proveedores ni productos relacionados.

5.5. Submenú 7.4: mantenimiento de productos (RF-JDBC-03)

MENÚ DE PRODUCTOS

1. Dar de alta un producto

2. Consultar producto por código

3. Modificar un producto

4. Eliminar un producto

0. Volver al menú MySQL

Funcionamiento y reglas de cada operación:

- **Alta de producto:** Solicitar código (1–12 caracteres: A–Z, 0–9 y guion; convertir a mayúsculas), nombre (3–100 caracteres), precio positivo con máximo dos decimales, stock inicial entero no negativo, ID de categoría e ID de proveedor. Comprobar que el código no existe y que ambas referencias sí existen en MySQL. Usar BigDecimal y PreparedStatement. Si cualquier dato es incorrecto, no insertar.
- **Consulta por código:** Pedir el código, normalizarlo y consultar MySQL. Mostrar código, nombre, precio, stock, IDs y, cuando sea posible, nombres de categoría y proveedor. Si no existe, informar sin lanzar una excepción al usuario.
- **Modificación:** Pedir código y mostrar los valores actuales. Permitir modificar nombre, precio, categoría y proveedor, indicando que pulsar Intro conserva cada valor. Validar nuevamente todos los campos modificados y la existencia de las claves foráneas. No permitir cambiar el código ni modificar directamente el stock: los cambios de existencias se realizan en el menú de movimientos.
- **Eliminación:** Solicitar código, mostrar los datos y pedir confirmación S/N. Si el producto tiene movimientos históricos, la clave foránea impedirá borrarlo: mostrar el motivo en rojo y conservar el historial. Nunca borrar movimientos para forzar la eliminación.

5.6. Submenú 7.5: consultas de productos (RF-JDBC-04)

MENÚ DE CONSULTAS

1. Listar todos los productos
2. Buscar producto por código
3. Buscar productos por nombre
4. Consultar productos con stock inferior a un umbral
0. Volver al menú MySQL

Funcionamiento y reglas de cada operación:

- **Listado completo:** Ejecutar SELECT con JOIN a categoria y proveedor, ordenado por código. Mostrar código, nombre, precio, stock, categoría y proveedor con encabezados legibles. Si no hay productos, mostrar un aviso informativo.
- **Búsqueda por código:** Solicitar código, normalizarlo y realizar una búsqueda exacta mediante PreparedStatement. Mostrar la ficha completa o indicar que no se han encontrado coincidencias.
- **Búsqueda por nombre:** Solicitar un fragmento de texto no vacío. Utilizar LIKE con parámetro preparado para buscar coincidencias parciales; cuando la colación de MySQL lo permita, la búsqueda no distinguirá mayúsculas y minúsculas. Mostrar todos los resultados, sin alterar los registros.
- **Stock inferior a un umbral:** Solicitar un número entero mayor o igual que cero y mostrar los productos cuyo stock sea estrictamente menor que ese valor (stock < umbral). Ordenar primero por stock y después por código. Si no hay coincidencias, informar sin mostrar un error.

5.7. Submenú 7.6: movimientos de stock (RF-JDBC-05)

MENÚ DE MOVIMIENTOS

1. Registrar una ENTRADA de stock
2. Registrar una SALIDA de stock
0. Volver al menú MySQL

Funcionamiento y reglas de cada operación:

- **Datos solicitados:** Pedir código de producto y comprobar que existe. Mostrar nombre y stock actual. Solicitar cantidad entera estrictamente positiva y confirmar la operación (S/N) antes de ejecutarla.
- **Ejecución JDBC:** Invocar el procedimiento almacenado registrar_movimiento(?, ?, ?) con CallableStatement, indicando código, tipo ENTRADA o SALIDA y cantidad. No actualizar producto.stock directamente desde Java en este menú.
- **Resultado correcto:** Consultar el nuevo stock y el movimiento registrado; mostrar stock anterior, cantidad, tipo y stock posterior. Utilizar [OK] en verde.
- **Errores y cancelaciones:** Si el producto no existe, la cantidad no es positiva o una SALIDA supera las existencias, mostrar [ERROR] en rojo. No deben modificarse ni el stock ni el historial. Si el usuario cancela, volver al submenú sin cambios.

5.8. Submenú 7.7: historial (RF-JDBC-06)

MENÚ DE HISTORIAL

1. Consultar todos los movimientos
2. Consultar movimientos de un producto
0. Volver al menú MySQL

Funcionamiento y reglas de cada operación:

- **Historial completo:** Consultar la tabla movimiento_stock y mostrar ID del movimiento, fecha y hora, código de producto, tipo, cantidad, stock anterior y stock posterior. Ordenar por fecha e ID de forma estable.
- **Historial por producto:** Solicitar código y comprobar que existe el producto. Mostrar únicamente sus movimientos, con los mismos campos y orden. Si existe pero no tiene movimientos, indicar «Sin movimientos registrados»; si no existe, informar expresamente.
- **Regla de consulta:** Las operaciones de historial son de solo lectura: no modifican el stock, no generan movimientos nuevos y no eliminan registros.

5.9. Opción 7.8: copiar inventario de memoria a MySQL (RF-JDBC-07)

OPCIÓN DE IMPORTACIÓN

8. Copiar productos del inventario en memoria a MySQL

Funcionamiento y reglas de cada operación:

- **Comprobación inicial:** Mostrar el número de productos que contiene la colección Java de la UT1. Si está vacía, indicar que no hay productos para importar y regresar. Si contiene registros, pedir confirmación S/N.
- **Validación individual:** Recorrer los productos y comprobar código, nombre, precio, stock, duplicidad del código en MySQL y existencia de la categoría y el proveedor. Los productos duplicados se omiten; los que incumplen otras reglas se rechazan, indicando el motivo de cada uno. No crear categorías o proveedores automáticamente.
- **Inserción y transacción:** Insertar los productos aceptados mediante PreparedStatement dentro de una única transacción JDBC para el lote: setAutoCommit(false), commit al completar las inserciones y rollback ante un error SQL inesperado. Los productos descartados por validación previa no provocan por sí solos un rollback.
- **Resumen final:** Mostrar los totales de productos insertados, omitidos por duplicado y rechazados por otras causas, con mensajes de éxito en verde y errores en rojo. Si se produce rollback, explicar que no se ha confirmado ninguna inserción del lote.
- **Conservación de datos:** La importación copia datos de la colección Java a MySQL: no borra ni modifica el inventario en memoria. Tampoco establece sincronización automática posterior entre ambos almacenes.

5.10. Opción 7.9: demostración de transacciones JDBC (RF-JDBC-08)

OPCIÓN DE TRANSACCIONES

9. Ejecutar demostración de commit y rollback

Funcionamiento y reglas de cada operación:

- **Finalidad y ubicación:** Esta demostración se ejecuta desde el menú MySQL de DataStock. Es una actividad técnica para practicar el control de transacciones desde Java; no representa una operación comercial del inventario.
- **Preparación:** Elegir y mostrar identificadores de categorías de prueba que no estén utilizados ni asociados a productos. Solicitar confirmación S/N antes de comenzar y explicar qué registros se van a insertar.
- **Escenario A: commit:** Desactivar autocommit con setAutoCommit(false), insertar dos categorías nuevas mediante PreparedStatement y ejecutar commit. Comprobar con SELECT que ambas existen y mostrar [OK] en verde.
- **Escenario B: rollback:** Iniciar otra transacción, insertar una categoría de prueba y provocar intencionadamente un error de clave duplicada en la segunda inserción. Capturar SQLException, ejecutar rollback y comprobar que la primera categoría del intento fallido tampoco permanece en MySQL. Mostrar el error esperado y explicar que el rollback ha funcionado correctamente.
- **Limpieza y cierre:** Restaurar autoCommit, cerrar los recursos y documentar los IDs utilizados y cómo eliminar únicamente los datos de demostración que no tengan productos asociados. No reutilizar IDs que puedan afectar a registros reales.

6. Requisitos funcionales de la UT2

RF-JDBC-01. Conectar y comprobar el esquema

Al entrar en la opción 7.1, DataStock abrirá una conexión a MySQL y comprobará que el esquema está disponible.

Qué tienes que desarrollar:

1. Mostrar confirmación de conexión, nombre de la base de datos y versión del gestor (DatabaseMetaData).
2. Comprobar con una consulta sencilla el acceso a las tablas y mostrar el número de productos.
3. Cerrar correctamente los recursos incluso cuando se produce una excepción.

Comprobaciones y situaciones de error:

- Servidor detenido, credenciales incorrectas o esquema inexistente: informar sin revelar la contraseña.
- La conexión no debe quedar abierta indefinidamente.

Pruebas de aceptación que debes registrar:

- CP-JDBC-01: conexión correcta; se muestran datos de la BD.
- CP-JDBC-02: contraseña incorrecta; mensaje controlado y aplicación continúa.

RF-JDBC-02. Gestionar categorías y proveedores

Permitir mantener los datos maestros necesarios para asociar productos a una categoría y a un proveedor existentes.

Qué tienes que desarrollar:

1. Categorías: alta (id, nombre), listado, modificación de nombre y baja por id.
2. Proveedores: alta (id, nombre, email opcional), listado, modificación de nombre/email y baja por id.
3. Comprobar el resultado de executeUpdate(): comunicar si se modificó una fila o no existía el registro.
4. Utilizar PreparedStatement para todos los valores recibidos del usuario.

Comprobaciones y situaciones de error:

- No aceptar identificadores no positivos ni nombres vacíos.
- Rechazar duplicados de id, nombre de categoría o email de proveedor sin finalizar el programa.
- Si existen productos relacionados, la base de datos puede impedir la baja: explicar la causa; no borrar productos automáticamente.

Pruebas de aceptación que debes registrar:

- CP-JDBC-03: alta y listado de categoría/proveedor.
- CP-JDBC-04: duplicado y baja con productos asociados.

RF-JDBC-03. CRUD de productos

Permitir altas, consultas, modificaciones y bajas de productos directamente en la tabla producto.

Qué tienes que desarrollar:

1. Alta: solicitar código, nombre, precio, stock, categoría y proveedor; comprobar datos y ejecutar INSERT preparado.
2. Modificación: permitir cambiar nombre, precio, categoría y proveedor; el stock se modifica únicamente mediante la funcionalidad de movimientos.
3. Baja: solicitar código y confirmación antes de ejecutar DELETE.
4. Consulta por código: mostrar el registro o indicar que no existe.

Comprobaciones y situaciones de error:

- Código en mayúsculas, de 1–12 caracteres, con letras A–Z, dígitos o guion.
- Precio BigDecimal positivo con dos decimales como máximo; stock inicial no negativo.
- Categoría/proveedor inexistente: explicar el problema de integridad referencial.

- Si hay movimientos históricos asociados, la baja debe ser rechazada por la FK; no eliminar el historial.

Pruebas de aceptación que debes registrar:

- CP-JDBC-05: insertar y recuperar producto con todos sus datos.
- CP-JDBC-06: modificar nombre y precio; comprobar en MySQL.
- CP-JDBC-07: intentar duplicar código y asociar categoría inexistente.
- CP-JDBC-08: baja permitida y baja bloqueada por movimientos.

RF-JDBC-04. Consultas con ResultSet

Realizar consultas SQL con filtros y mostrar resultados legibles.

Qué tienes que desarrollar:

1. Listar productos ordenados por código incluyendo nombres de categoría y proveedor mediante JOIN.
2. Buscar por código exacto y por fragmento de nombre con LIKE y parámetros preparados.
3. Mostrar productos con stock por debajo de un umbral indicado.
4. Construir objetos Java o DTO a partir de ResultSet, evitando presentar la fila como un conjunto de valores sin identificar.

Comprobaciones y situaciones de error:

- Si no hay coincidencias, mostrar un mensaje y volver al menú.
- No concatenar entradas del usuario directamente en SQL.
- Los filtros no deben alterar registros.

Pruebas de aceptación que debes registrar:

- CP-JDBC-09: consulta con JOIN y orden.
- CP-JDBC-10: búsqueda parcial y consulta sin resultados.
- CP-JDBC-11: umbral de stock.

RF-JDBC-05. Registrar movimientos con procedimiento almacenado

Usar el procedimiento registrar_movimiento incluido en el SQL para efectuar entradas y salidas atómicas de stock.

Qué tienes que desarrollar:

1. Solicitar código de producto, tipo ENTRADA/SALIDA y cantidad positiva.
2. Invocar CALL registrar_movimiento(?, ?, ?) desde Java mediante CallableStatement.
3. Tras la operación, consultar el stock actual y el último movimiento y mostrar stock anterior y posterior.
4. No actualizar producto.stock directamente desde el menú de movimientos: la lógica transaccional la ejecuta el procedimiento.

Comprobaciones y situaciones de error:

- Producto inexistente, cantidad 0/negativa, tipo no admitido o salida superior al stock: no debe cambiar ni el producto ni el historial.
- El procedimiento incluye START TRANSACTION, SELECT FOR UPDATE, UPDATE, INSERT, COMMIT y ROLLBACK ante error; no añadas una segunda transacción exterior que pueda interferir con él.

Pruebas de aceptación que debes registrar:

- CP-JDBC-12: entrada de 5 unidades y comprobación de ambos registros.
- CP-JDBC-13: salida válida y cálculo de stock.
- CP-JDBC-14: salida superior al stock; comprobar que no aparece movimiento nuevo.

```
Producto TEC-001: stock inicial 8
Movimiento: ENTRADA, cantidad 5
```

```
Resultado: stock anterior 8 -> posterior 13
Movimiento: SALIDA, cantidad 4
Resultado: stock anterior 13 -> posterior 9
Movimiento: SALIDA, cantidad 20
ERROR: Stock insuficiente. El stock permanece en 9.
```

RF-JDBC-06. Consultar historial de movimientos

Mostrar los movimientos almacenados, sin confundir el stock actual con el histórico.

Qué tienes que desarrollar:

1. Permitir consultar todos los movimientos y filtrar por código de producto.
2. Mostrar id, fecha, tipo, cantidad, stock_anterior y stock_posterior, ordenados por fecha e id.
3. Diferenciar los movimientos de los productos que actualmente no tengan movimientos.

Comprobaciones y situaciones de error:

- No modificar el historial al consultar.
- Gestionar productos sin movimientos.

Pruebas de aceptación que debes registrar:

- CP-JDBC-15: historial ordenado y filtrado.
- CP-JDBC-16: producto sin movimientos.

RF-JDBC-07. Importación controlada desde la UT1

Aprovechar el inventario en memoria de la UT1 para persistir en MySQL los productos válidos que todavía no existan.

Qué tienes que desarrollar:

1. Ofrecer una operación explícita «Importar productos de memoria a MySQL»; no sincronizar automáticamente.
2. Comprobar previamente categorías y proveedores existentes, códigos repetidos y demás restricciones de MySQL.
3. Insertar los productos aceptados mediante PreparedStatement e informar de insertados, omitidos y rechazados con sus motivos.
4. Establecer una transacción JDBC con setAutoCommit(false): si falla una operación SQL inesperada durante el lote, rollback de todas las inserciones de ese lote; si termina correctamente, commit.

Comprobaciones y situaciones de error:

- Los registros rechazados por validación previa no se intentan insertar; el lote puede incluir otros registros válidos.
- Si se produce un fallo SQL durante el lote, no debe quedar insertada una parte del lote.
- No alterar el inventario en memoria de UT1 por el hecho de importar.

Pruebas de aceptación que debes registrar:

- CP-JDBC-17: lote con válidos, duplicados y referencias inexistentes; resumen correcto.
- CP-JDBC-18: fallo SQL forzado; rollback sin inserciones parciales.

RF-JDBC-08. Transacción JDBC de demostración

Demostrar expresamente que sabes controlar commit y rollback desde Java, además del procedimiento almacenado.

Qué tienes que desarrollar:

1. Crear una operación de demostración que, dentro de una transacción JDBC, inserte dos categorías de prueba con ids no usados; confirmar ambas con commit.
2. Repetir con una segunda inserción intencionadamente duplicada: debe ejecutarse rollback y no permanecer la primera inserción del intento fallido.
3. Restaurar el valor de autoCommit y cerrar recursos. Documentar los ids usados y cómo limpiar los datos de prueba.

Comprobaciones y situaciones de error:

- No usar las tablas reales con ids que puedan afectar a productos existentes.
- No confundir el rollback Java de este ejercicio con el que ya implementa el procedimiento almacenado.

Pruebas de aceptación que debes registrar:

- CP-JDBC-19: dos inserciones correctas y commit.
- CP-JDBC-20: duplicado y rollback total.

5.11. Ejemplo completo de recorrido de usuario

En este apartado se presenta un ejemplo práctico del funcionamiento esperado de DataStock una vez incorporadas las funcionalidades de la UT2.

El objetivo es comprender cómo se relacionan las operaciones desarrolladas en la UT1, que trabajan con productos almacenados en memoria, con las nuevas funcionalidades JDBC, que permiten guardar y gestionar esos productos de forma permanente en MySQL.

El ejemplo describe una posible secuencia de utilización de la aplicación. Los mensajes mostrados son orientativos: no es necesario reproducirlos literalmente, pero sí implementar las operaciones, validaciones y comportamientos indicados.

6.1.1. Situación inicial

Supongamos que disponemos de un fichero JSON que contiene tres productos:

Código	Nombre	Precio	Stock	ID categoría	ID proveedor
TEC-001	Teclado mecánico	49,90 €	10	1	1
RAT-002	Ratón inalámbrico	19,95 €	25	1	1
MON-003	Monitor 24 pulgadas	149,00 €	8	2	3

Inicialmente, la aplicación acaba de ejecutarse y el inventario en memoria está vacío.

Para este ejemplo, supondremos también que las tablas de MySQL están creadas, pero todavía no contienen categorías, proveedores ni productos.

Importante: el script DataStock_BD_MySQL.sql crea la estructura de la base de datos, pero no introduce automáticamente los datos necesarios para este ejemplo. Por tanto, tendremos que dar de alta las categorías y proveedores antes de guardar los productos.

6.1.2. Paso 1. Importar los productos desde un fichero JSON (UT1)

Desde el menú principal de DataStock, seleccionamos la opción 2. Importar productos desde JSON

La aplicación solicitará la ruta del fichero que contiene los productos.

Una vez localizado y procesado correctamente, los objetos Producto se incorporarán al inventario en memoria, utilizando las funcionalidades desarrolladas durante la UT1.

Ejemplo de interacción:

```
================================================
         DATASTOCK - GESTIÓN DE INVENTARIO
================================================

1. Explorar ficheros y directorios
```

```
2. Importar productos desde JSON
3. Importar productos desde XML
4. Consultar productos en memoria
5. Exportar productos
6. Guardar y recuperar inventario
7. Gestionar base de datos MySQL

0. Salir

Selecciona una opción: 2

Ruta del fichero JSON: datos/productos.json

[OK] Fichero JSON procesado correctamente.

Productos importados: 3
Productos disponibles en memoria: 3
```

En este momento:

- Los tres productos se encuentran almacenados en una colección Java.
- Podemos consultarlos utilizando la opción 4 del menú principal.
- Todavía no se ha insertado ninguno de estos productos en MySQL.
- Si cerramos la aplicación sin guardar el inventario en un fichero, los productos dejarán de estar disponibles en memoria.

Comprobación: el alumno deberá verificar que los tres productos aparecen en las consultas del inventario en memoria.

6.1.3. Paso 2. Acceder a las funcionalidades de MySQL (UT2)

Desde el menú principal seleccionamos: **7. Gestionar base de datos MySQL**

La aplicación mostrará el nuevo menú desarrollado en esta unidad.

```
================================================
              GESTIÓN MYSQL - UT2
================================================

1. Comprobar conexión y mostrar información de la BD
2. Gestionar categorías
3. Gestionar proveedores
4. Gestionar productos (CRUD)
5. Consultar y buscar productos
6. Registrar entrada o salida de stock
7. Consultar historial de movimientos
8. Importar productos de memoria a MySQL
9. Demostración de transacción con commit/rollback

0. Volver al menú principal
```

Antes de comenzar a insertar información, seleccionamos la opción **1. Comprobar conexión y mostrar información de la BD**.

La aplicación deberá intentar conectarse a MySQL utilizando la configuración JDBC establecida.

Si la conexión es correcta, mostrará un mensaje de confirmación en verde, junto con la información solicitada en el requisito RF-JDBC-01.

```
Selecciona una opción: 1

[OK] Conexión establecida correctamente.
```

```
Base de datos: datastock
Gestor: MySQL
Versión: [versión detectada]
Productos almacenados: 0
```

Si la conexión falla, deberá mostrarse un mensaje explicativo en rojo, sin finalizar bruscamente la aplicación.

Comprobación: el alumno deberá verificar que puede establecer la conexión y consultar la base de datos antes de continuar.

6.1.4. Paso 3. Dar de alta las categorías necesarias

Los productos importados desde JSON hacen referencia a las categorías con identificadores 1 y 2. Como estas categorías todavía no existen en MySQL, debemos crearlas antes de importar los productos. Desde el menú de gestión MySQL seleccionamos: **2. Gestionar categorías → 1. Alta categoría**

Introducimos los siguientes datos:

Primera categoría:

```
ALTA DE CATEGORÍA
----------------------------------------
ID de categoría: 1
Nombre: Periféricos

[OK] Categoría registrada correctamente.
```

Segunda categoría:

```
ALTA DE CATEGORÍA
----------------------------------------
ID de categoría: 2
Nombre: Monitores

[OK] Categoría registrada correctamente.
```

A continuación, utilizamos la opción **Listar categorías** para comprobar que ambas se encuentran almacenadas en MySQL.

```
LISTADO DE CATEGORÍAS
----------------------------------------

ID    NOMBRE
----------------------------------------
1     Periféricos
2     Monitores
```

Comprobación: las categorías deben aparecer en el listado y existir realmente en la tabla categoria.

6.1.5. Paso 4. Dar de alta el proveedor necesario

Los productos TEC-001 y RAT-002 hacen referencia al proveedor con identificador 1. Desde el menú de gestión MySQL seleccionamos: **3. Gestionar proveedores → 1. Alta proveedor**

Introducimos los siguientes datos:

```
ALTA DE PROVEEDOR
----------------------------------------

ID de proveedor: 1
Nombre: Distribuciones Norte
Email (opcional): ventas@distribuciones.example

[OK] Proveedor registrado correctamente.
```

A continuación, consultamos el listado de proveedores para verificar que se ha almacenado.

En este ejemplo **no daremos de alta el proveedor con identificador 3**, al que hace referencia el producto MON-003.

Esto nos permitirá comprobar cómo debe comportarse DataStock cuando intentamos importar un producto cuya clave foránea no existe en MySQL.

6.1.6. Paso 5. Importar los productos almacenados en memoria a MySQL

Una vez creadas las categorías y el proveedor necesarios, regresamos al menú de gestión MySQL y seleccionamos: **8. Importar productos de memoria a MySQL**

Esta opción constituye el punto de integración entre la UT1 y la UT2. La aplicación deberá recuperar los productos que ya se encuentran en la colección Java e intentar almacenarlos en la tabla producto mediante JDBC.

Antes de comenzar, mostrará el número de productos disponibles y solicitará confirmación.

```
IMPORTAR PRODUCTOS DE MEMORIA A MYSQL
----------------------------------------------
Productos disponibles en memoria: 3
¿Deseas importar estos productos a MySQL? (S/N): S
Iniciando comprobación de productos...
```

Para cada producto, DataStock deberá realizar las validaciones establecidas en el apartado 3.1 y en el requisito RF-JDBC-07. En particular, comprobará:

- Que el código cumple el formato permitido y no existe ya en MySQL.
- Que el nombre, precio y stock son válidos.
- Que la categoría indicada existe en la tabla categoria.
- Que el proveedor indicado existe en la tabla proveedor.

Resultado esperado para cada producto:

Producto TEC-001:

- Su código y restantes datos son válidos.
- La categoría 1 existe.
- El proveedor 1 existe.
- El código no está duplicado.

Resultado: el producto puede insertarse correctamente en MySQL.

Producto RAT-002:

- Su código y restantes datos son válidos.
- La categoría 1 existe.
- El proveedor 1 existe.
- El código no está duplicado.

Resultado: el producto puede insertarse correctamente en MySQL.

Producto MON-003:

- Sus datos individuales son válidos.
- La categoría 2 existe.

- El proveedor 3 no existe en MySQL.

Resultado: el producto debe rechazarse por incumplir la restricción de integridad referencial.

La aplicación podría mostrar:

```
IMPORTAR PRODUCTOS DE MEMORIA A MYSQL
------------------------------------------------
Comprobando TEC-001 - Teclado mecánico...
[OK] Producto válido para importar.

Comprobando RAT-002 - Ratón inalámbrico...
[OK] Producto válido para importar.

Comprobando MON-003 - Monitor 24 pulgadas...
[ERROR] No existe el proveedor con ID 3.
        El producto no se puede importar.



------------------------------------------------
RESULTADO DE LA IMPORTACIÓN
------------------------------------------------

Productos analizados:              3
Productos insertados:              2
Omitidos por código duplicado:     0
Rechazados por datos incorrectos:  1

[OK] Importación finalizada.

Los productos válidos se han guardado en MySQL.
```

Los mensajes [OK] se mostrarán en verde y los mensajes [ERROR] en rojo.

Importante: que un producto sea rechazado por una validación previa no debe impedir que se importen los demás productos válidos.

Las inserciones aceptadas se realizarán dentro de una transacción JDBC. Si durante la ejecución del lote se produce un error SQL inesperado, deberá ejecutarse rollback() para evitar que quede almacenada solo una parte de las inserciones previstas.

Al finalizar correctamente el lote, se ejecutará commit().

Comprobación: al terminar esta operación, MySQL debe contener los productos TEC-001 y RAT-002, mientras que MON-003 seguirá estando únicamente en el inventario en memoria.

6.1.7. Paso 6. Consultar los productos almacenados en MySQL

Ahora seleccionamos la opción 5. Consultar y buscar productos → 1. Listar todos

La aplicación deberá ejecutar una consulta SQL sobre MySQL y mostrar los productos almacenados.

En el listado se incluirán los nombres de las categorías y proveedores, utilizando las relaciones entre las tablas.

Ejemplo de resultado:

```
PRODUCTOS ALMACENADOS EN MYSQL
------------------------------------------------------------
Código: TEC-001
Nombre: Teclado mecánico
Precio: 49.90 €
Stock: 10
Categoría: Periféricos
```

```
Proveedor: Distribuciones Norte
------------------------------------------------------------
Código: RAT-002
Nombre: Ratón inalámbrico
Precio: 19.95 €
Stock: 25
Categoría: Periféricos
Proveedor: Distribuciones Norte
------------------------------------------------------------
Total de productos encontrados: 2
```

Comprobación: el listado debe mostrar únicamente los dos productos insertados en MySQL.

Aunque el inventario en memoria contiene tres productos, las consultas JDBC no deben utilizar esa colección, sino recuperar la información directamente de la base de datos.

6.1.8. Paso 7. Cerrar DataStock y volver a ejecutar la aplicación

Este paso es fundamental para comprobar que los datos se han almacenado de forma permanente.

Primero, cerramos completamente DataStock utilizando la opción **0. Salir** del menú principal. Después, volvemos a ejecutar la aplicación. Como hemos iniciado una nueva ejecución, el inventario en memoria estará inicialmente vacío, salvo que la aplicación tenga implementada alguna carga automática de datos.

Seleccionamos: **4. Consultar productos en memoria**

El resultado esperado será:

```
CONSULTA DEL INVENTARIO EN MEMORIA
----------------------------------------
No hay productos cargados en memoria.
```

A continuación, accedemos a: **7. Gestionar base de datos MySQL → 5. Consultar y buscar productos → 1. Listar todos**

La aplicación deberá volver a mostrar los productos TEC-001 y RAT-002.

Esto demuestra que los datos permanecen almacenados en MySQL, aunque la colección Java se haya perdido al finalizar la ejecución anterior.

Comprobación: el alumno deberá verificar que el contenido de MySQL se mantiene entre ejecuciones y que no depende de que los productos estén cargados en memoria.

6.1.9. Paso 8. Completar los datos pendientes y repetir la importación

Finalmente, vamos a comprobar qué sucede cuando resolvemos el problema que había impedido importar el tercer producto. En primer lugar, damos de alta el proveedor que faltaba: **7. Gestionar base de datos MySQL → 3. Gestionar proveedores → 1. Alta proveedor**

```
ALTA DE PROVEEDOR
----------------------------------------
ID de proveedor: 3
Nombre: Pantallas del Centro
Email (opcional): contacto@pantallas.example

[OK] Proveedor registrado correctamente.
```

Después, como acabamos de reiniciar DataStock, volvemos a importar el fichero JSON desde la opción 2 del menú principal. El inventario en memoria vuelve a contener los tres productos.

Seleccionamos nuevamente: **7. Gestionar base de datos MySQL → 8. Importar productos de memoria a MySQL**

Ahora la aplicación deberá detectar lo siguiente:

- TEC-001: ya existe en MySQL. Se omite por código duplicado.

- RAT-002: ya existe en MySQL. Se omite por código duplicado.
- MON-003: ahora cumple todas las condiciones, porque la categoría 2 y el proveedor 3 existen. Se inserta correctamente.

El resultado esperado será:

```
IMPORTAR PRODUCTOS DE MEMORIA A MYSQL
------------------------------------------------

Comprobando TEC-001...
[AVISO] El producto ya existe en MySQL. Se omite.

Comprobando RAT-002...
[AVISO] El producto ya existe en MySQL. Se omite.

Comprobando MON-003...
[OK] Producto válido para importar.

------------------------------------------------
RESULTADO DE LA IMPORTACIÓN
------------------------------------------------

Productos analizados:              3
Productos insertados:              1
Omitidos por código duplicado:     2
Rechazados por datos incorrectos:  0

[OK] Importación finalizada.
```

Al consultar nuevamente los productos de MySQL, deberán aparecer los tres.

6.1.10. Comprobaciones finales del recorrido

Una vez completado el ejemplo, el alumno deberá verificar que se cumplen las siguientes condiciones:

- **Continuidad con la UT1:** la aplicación conserva la importación de productos desde JSON y XML y permite consultar el inventario en memoria.
- **Separación entre memoria y MySQL:** importar un fichero no guarda automáticamente los productos en la base de datos.
- **Gestión de datos relacionados:** las categorías y proveedores necesarios deben existir antes de insertar productos.
- **Validación de la información:** los productos que incumplen alguna restricción no se almacenan en MySQL.
- **Control de duplicados:** repetir una importación no genera registros con códigos repetidos ni sobrescribe los productos existentes.
- **Persistencia:** los productos guardados en MySQL siguen disponibles después de cerrar y volver a ejecutar DataStock.
- **Uso de JDBC:** las inserciones y consultas se realizan mediante las clases y métodos JDBC correspondientes, no utilizando directamente la colección en memoria.
- **Tratamiento de errores:** la aplicación informa de los problemas mediante mensajes comprensibles en rojo y confirma las operaciones correctas en verde.
- **Integridad de los datos:** las operaciones respetan las restricciones definidas en el script SQL proporcionado.

Recuerda: este recorrido es un ejemplo de utilización de DataStock, no una funcionalidad adicional que deba programarse de manera independiente. Todas las operaciones descritas deben poder realizarse utilizando los menús y submenús desarrollados en las UT1 y UT2.

El objetivo es demostrar que la aplicación ha evolucionado desde un inventario temporal, gestionado mediante objetos Java y ficheros, hasta un sistema capaz de almacenar y consultar información de forma persistente mediante JDBC y MySQL.

7. Actividad comparativa obligatoria: gestor embebido e independiente

Esta actividad es OBLIGATORIA para evidenciar RA2-b y se realiza EN EL MISMO PROYECTO MAVEN DataStock, pero como una clase Java ejecutable independiente (por ejemplo, es.dam.datastock.complementarias.DemoSQLite). No se añade ninguna opción al menú principal ni al submenú MySQL. No debes convertir DataStock a SQLite: la aplicación principal sigue utilizando MySQL.

7.1. Instalación y dependencia JDBC SQLite

SQLite es un motor embebido: la base de datos se almacena en un fichero local y no requiere instalar un servidor.

Consulta la página oficial https://www.sqlite.org/download.html (descargas y herramientas opcionales) y la documentación https://www.sqlite.org/quickstart.html.

Para esta actividad Java basta con añadir al **pom.xml** el **controlador JDBC org.xerial:sqlite-jdbc** (consulta su documentación oficial: https://github.com/xerial/sqlite-jdbc); Maven descargará la biblioteca.

No es obligatorio instalar el ejecutable sqlite3 ni DB Browser para completar la actividad. Indica en el README la versión concreta elegida.

7.2. Base de datos de demostración y tabla

Crea el fichero SQLite en una ruta local del proyecto, por ejemplo datos/sqlite/productos_demo.db (exclúyelo de Git si contiene resultados personales). JDBC URL: jdbc:sqlite:datos/sqlite/productos_demo.db. Crea previamente el directorio datos/sqlite desde Java si no existe. El fichero se creará al abrir la conexión si la ruta es válida.

```
CREATE TABLE IF NOT EXISTS producto_demo (
  codigo TEXT PRIMARY KEY NOT NULL,
  nombre TEXT NOT NULL
);
```

Esta tabla es una simplificación didáctica y NO sustituye a las cuatro tablas MySQL de DataStock. No necesita categorías, proveedores ni movimientos.

7.3. Desarrollo paso a paso de DemoSQLite

1. Añade la dependencia sqlite-jdbc al pom.xml y crea la clase ejecutable DemoSQLite con su método main.
2. Abre una conexión JDBC SQLite con DriverManager.getConnection(url).
3. Crea la tabla mediante Statement y CREATE TABLE IF NOT EXISTS.
4. Inserta mediante PreparedStatement dos registros de ejemplo: (SQL-001, Producto demo A) y (SQL-002, Producto demo B).
5. Ejecuta SELECT codigo, nombre FROM producto_demo ORDER BY codigo, recorre ResultSet y muestra ambos campos.
6. Usa try-with-resources para Connection, Statement, PreparedStatement y ResultSet.
7. Repite la ejecución y controla el duplicado: puedes usar INSERT OR IGNORE e indicar las filas realmente insertadas; no deben acumularse copias.
8. Comprueba que el fichero .db persiste tras finalizar el programa.

```
Ejemplo de salida:
[OK] Conexión SQLite abierta: datos/sqlite/productos_demo.db
[OK] Tabla producto_demo disponible.
Insertados: 2 | Ya existentes: 0
SQL-001 | Producto demo A
SQL-002 | Producto demo B
```

7.4. Comparación escrita con MySQL

En el README, añade una sección «Comparativa MySQL y SQLite» con una tabla que responda:

- ¿necesita servidor?,
- ¿dónde se guardan los datos?,
- ¿cómo se establece la conexión JDBC?,
- ¿qué dependencia se utiliza?,
- ¿qué ventajas e inconvenientes tiene cada uno?,
- ¿para qué tipo de aplicación lo elegirías?
- Incluye además una breve explicación de qué es un pool de conexiones, cuándo resulta útil y por qué DataStock puede funcionar con conexiones cortas y bien cerradas.

7.5. Pruebas y evidencias de la actividad

Registra CP-SQLITE-01: primera ejecución, creación del fichero y dos filas recuperadas; CP-SQLITE-02: segunda ejecución, no se duplican registros; CP-SQLITE-03: comprobación del cierre de recursos y fichero persistente. Conserva el código y la explicación en el mismo repositorio, y documenta en README cómo ejecutar la clase DemoSQLite por separado. No se exige integrar SQLite en los menús DataStock.

8. Reglas técnicas y de seguridad

- Java 21, Maven, JDBC y sentencias preparadas; no utilizar Hibernate/JPA en UT2.
- Credenciales fuera del repositorio. README con instrucciones para configurar variables de entorno JDBC_URL, JDBC_USER y JDBC_PASSWORD, o un properties local excluido de Git; no subir archivos con secretos.
- Cerrar conexiones, sentencias y resultados con try-with-resources.
- Mensajes de error útiles, sin mostrar contraseñas ni trazas técnicas completas al usuario final.
- Separar al menos la interacción de consola de las operaciones JDBC; la formalización en componentes e interfaces se exigirá en UT3.
- Javadoc y comentarios que expliquen decisiones, no comentarios redundantes que repitan cada instrucción.
- Probar operaciones en MySQL Workbench para verificar que el estado de la BD coincide con lo mostrado por la consola.

9. Evaluación y relación con los criterios de evaluación

UT2 evalúa el RA2 completo, con once criterios (a–k). En la programación didáctica, el RA2 representa el 20 % de la calificación del módulo. La práctica aporta evidencias de los criterios, junto con el resto de instrumentos previstos en la programación; no se debe interpretar que esta entrega, por sí sola, supone el 20 % del módulo.

CE	Qué se comprobará en esta entrega
RA2-a	Comparación razonada de conectores y uso de JDBC.
RA2-b	MySQL independiente y ejercicio SQLite embebido.
RA2-c	Elección y configuración de Connector/J y driver SQLite.
RA2-d	Conexión correcta y tratamiento de errores.
RA2-e	Creación y verificación del esquema SQL proporcionado.
RA2-f	Altas, modificaciones, bajas e importación a BD.
RA2-g	Lectura de ResultSet y transformación en objetos.
RA2-h	Consultas SQL, JOIN y filtros.
RA2-i	Cierre de Connection, Statement y ResultSet.

RA2-j	Commit y rollback desde JDBC.
RA2-k	Invocación de registrar_movimiento con CallableStatement.

La profesora podrá comprobar individualmente el funcionamiento, pedir explicaciones sobre el código o solicitar la reproducción de una prueba. La autoría y comprensión son necesarias para valorar adecuadamente la evidencia presentada.

10. Entregables de la segunda fase

1. Repositorio GitHub individual: continuidad del mismo proyecto de UT1, código Maven actualizado y funcionalidades UT1 conservadas.
2. Script SQL proporcionado en una carpeta de referencia y, en su caso, scripts propios de datos de prueba separados. No subir exportaciones que contengan datos personales ni contraseñas.
3. Clase o actividad SQLite de comparación, con breve explicación técnica en README.
4. README actualizado: requisitos, configuración sin secretos, instalación del esquema, ejecución, menús, estructura y diferencias memoria/MySQL.
5. Documento de pruebas ampliado con CP-JDBC-01 a CP-JDBC-20 y CP-SQLITE-01 a CP-SQLITE-03, indicando datos, resultado esperado, obtenido y evidencia. Entregar también PDF en Moodle.
6. Historial Git con commits significativos y etiqueta UT2-v1.0 sobre la versión presentada.
7. En el buzón Moodle: enlace al repositorio, referencia a la etiqueta UT2-v1.0 y PDF del registro de pruebas.

11. Lista final de comprobación de la entrega UT2

Antes de realizar la segunda entrega de DataStock, deberás comprobar que tu aplicación funciona correctamente y que has implementado todas las funcionalidades solicitadas.

Esta lista de comprobación te permitirá revisar tu trabajo de manera ordenada, detectar posibles errores y verificar que la aplicación cumple los requisitos de la UT2.

No es suficiente con que el proyecto compile o que los menús aparezcan en pantalla. Debes comprobar que las operaciones se ejecutan correctamente, que los datos se almacenan en MySQL y que la aplicación responde adecuadamente cuando se producen errores.

Marca cada casilla únicamente cuando hayas realizado la comprobación correspondiente.

11.1. Continuidad del proyecto y funcionalidades de la UT1

La UT2 constituye una ampliación del proyecto desarrollado en la UT1. Por tanto, las nuevas funcionalidades deben integrarse en la misma aplicación, conservando las anteriores.

Comprueba lo siguiente:

- ☐ He continuado trabajando sobre el proyecto Maven de DataStock utilizado en la UT1, sin crear una aplicación independiente para MySQL.
- ☐ El proyecto se compila y ejecuta correctamente con Java 21.
- ☐ Se mantienen las clases del modelo y las utilidades desarrolladas en la UT1.
- ☐ Se conservan las opciones originales del menú principal.

- ☐ Puedo explorar ficheros y directorios.
- ☐ Puedo importar productos desde un fichero JSON.
- ☐ Puedo importar productos desde un fichero XML.
- ☐ Los productos importados se incorporan correctamente al inventario en memoria.
- ☐ Puedo consultar los productos almacenados en memoria.
- ☐ Puedo exportar los productos a los formatos solicitados en la UT1.
- ☐ Puedo guardar y recuperar el inventario mediante serialización.
- ☐ Las funcionalidades de XPath y RandomAccessFile desarrolladas en la UT1 continúan disponibles y funcionando.
- ☐ La incorporación de JDBC no ha provocado errores en las funcionalidades anteriores.
- ☐ Comprendo que el inventario en memoria y la base de datos MySQL son almacenes diferentes.
- ☐ He comprobado que importar un fichero JSON o XML no provoca automáticamente la inserción de sus productos en MySQL.

Prueba recomendada: ejecuta DataStock, importa un fichero JSON con varios productos y comprueba que aparecen en memoria, pero no en MySQL, hasta que utilices expresamente la opción de importación a la base de datos.

11.2. Preparación de MySQL y configuración JDBC

Antes de comprobar las operaciones sobre la base de datos, debes verificar que el entorno está correctamente preparado.

Comprueba lo siguiente:

- ☐ He ejecutado el fichero DataStock_BD_MySQL.sql proporcionado con la práctica.
- ☐ Existe la base de datos datastock.
- ☐ Existen las cuatro tablas: categoria, proveedor, producto y movimiento_stock.
- ☐ Existe el procedimiento almacenado registrar_movimiento.
- ☐ No he eliminado ni modificado las restricciones de integridad definidas en el script SQL.
- ☐ He añadido al fichero pom.xml la dependencia correspondiente a MySQL Connector/J.
- ☐ Maven descarga y reconoce correctamente la dependencia JDBC.
- ☐ He implementado una clase o mecanismo reutilizable para establecer conexiones con MySQL.
- ☐ Los datos de conexión se obtienen de una configuración local o de variables de entorno.
- ☐ No he escrito contraseñas directamente en el código fuente.
- ☐ Los ficheros locales que contienen credenciales están excluidos del repositorio mediante .gitignore.
- ☐ La opción **7.1. Comprobar conexión** establece correctamente la conexión.
- ☐ Se muestra el nombre de la base de datos y la versión del gestor.
- ☐ La aplicación puede consultar el número de productos almacenados.
- ☐ Si MySQL no está disponible, se muestra un mensaje de error comprensible.
- ☐ Si las credenciales son incorrectas, la aplicación informa del problema sin mostrar la contraseña.

- ☐ Un fallo de conexión no provoca el cierre inesperado de DataStock.

Prueba recomendada: comprueba primero una conexión correcta. Después, detén temporalmente el servidor MySQL o utiliza una configuración incorrecta y verifica que la aplicación controla el error.

11.3. Menús, navegación y presentación de resultados

Todas las funcionalidades de la UT2 deben estar disponibles desde los menús de DataStock.

Comprueba lo siguiente:

- ☐ El menú principal incorpora la opción **7. Gestionar base de datos MySQL**.
- ☐ La opción 7 permite acceder al menú de funcionalidades JDBC.
- ☐ El menú MySQL contiene las nueve opciones funcionales especificadas en el apartado 5.
- ☐ Los submenús de categorías, proveedores, productos, consultas, movimientos e historial muestran las operaciones correspondientes.
- ☐ Todas las opciones del menú ejecutan la funcionalidad indicada.
- ☐ La opción 0 permite regresar al menú anterior o salir, según corresponda.
- ☐ Introducir una opción de menú inexistente no provoca una excepción ni finaliza la aplicación.
- ☐ Después de completar una operación, puedo continuar trabajando desde el menú correspondiente.
- ☐ Los datos se solicitan de manera clara, indicando qué debe introducir el usuario.
- ☐ Se informa cuando una operación requiere confirmación.
- ☐ Las eliminaciones solicitan confirmación mediante una respuesta S/N.
- ☐ Si el usuario cancela una operación, no se realizan modificaciones en la base de datos.
- ☐ Los mensajes de error aparecen en **rojo**.
- ☐ Los mensajes de operaciones completadas correctamente aparecen en **verde**.
- ☐ Se utiliza RESET después de aplicar colores ANSI para evitar que el resto de la consola mantenga el color anterior.
- ☐ Los mensajes permiten identificar el resultado de la operación incluso cuando la terminal no admite colores ANSI.
- ☐ Los listados presentan encabezados y datos legibles.
- ☐ Cuando una consulta no devuelve resultados, aparece un mensaje informativo y no un error de ejecución.

Prueba recomendada: recorre todas las opciones de los menús e introduce intencionadamente valores incorrectos para comprobar que la navegación y los mensajes funcionan correctamente.

11.4. Gestión de categorías

Desde la opción **7.2. Gestionar categorías**, debes poder realizar las operaciones de alta, consulta, modificación y eliminación.

Comprueba lo siguiente:

- ☐ Puedo dar de alta una categoría indicando su identificador y nombre.
- ☐ El identificador debe ser un número entero positivo.
- ☐ El nombre es obligatorio y tiene una longitud máxima de 60 caracteres.
- ☐ Se eliminan los espacios innecesarios al principio y al final del nombre.
- ☐ No puedo insertar una categoría con un identificador ya existente.

- ☐ No puedo insertar dos categorías con el mismo nombre.
- ☐ Después de un alta correcta, la categoría aparece en MySQL.
- ☐ Puedo listar todas las categorías ordenadas por identificador.
- ☐ Puedo modificar el nombre de una categoría existente.
- ☐ No se permite modificar el identificador de una categoría.
- ☐ Si intento modificar una categoría inexistente, se muestra un mensaje explicativo.
- ☐ Antes de eliminar una categoría, se muestran sus datos y se solicita confirmación.
- ☐ Puedo eliminar una categoría que no tenga productos asociados.
- ☐ No puedo eliminar una categoría que tenga productos asociados.
- ☐ Cuando MySQL impide una eliminación por una clave foránea, la aplicación muestra un mensaje de error en rojo.
- ☐ Una operación fallida no modifica otros registros.

11.5. Gestión de proveedores

Desde la opción **7.3. Gestionar proveedores**, debes poder mantener la información de los proveedores.

Comprueba lo siguiente:

- ☐ Puedo dar de alta un proveedor indicando identificador, nombre y correo electrónico.
- ☐ El identificador es un número entero positivo.
- ☐ El nombre es obligatorio y no supera los 100 caracteres.
- ☐ El correo electrónico es opcional.
- ☐ Si se introduce un correo, se comprueba que tiene un formato básico válido.
- ☐ El correo no supera los 120 caracteres.
- ☐ Si no se introduce correo, puede almacenarse como NULL.
- ☐ No puedo registrar dos proveedores con el mismo identificador.
- ☐ No puedo registrar dos proveedores con el mismo correo electrónico no nulo.
- ☐ Puedo listar los proveedores mostrando identificador, nombre y correo.
- ☐ Puedo modificar el nombre y el correo de un proveedor existente.
- ☐ Al modificar un proveedor, puedo conservar los valores actuales.
- ☐ Puedo dejar sin valor el correo electrónico utilizando el mecanismo establecido en el enunciado.
- ☐ Puedo eliminar un proveedor sin productos asociados.
- ☐ No puedo eliminar un proveedor que tenga productos asociados.
- ☐ Los errores de duplicidad o integridad referencial se muestran de manera comprensible.

11.6. Gestión de productos

Desde la opción **7.4. Gestionar productos**, debes poder realizar las operaciones CRUD directamente sobre MySQL.

Comprueba lo siguiente:

Alta de productos

- ☐ Puedo dar de alta un producto solicitando código, nombre, precio, stock inicial, categoría y proveedor.
- ☐ El código contiene entre 1 y 12 caracteres.
- ☐ El código solo contiene letras mayúsculas, números y guiones.
- ☐ Si se introducen letras minúsculas, se normalizan según lo establecido en el enunciado.
- ☐ No puedo insertar un producto cuyo código ya exista en MySQL.
- ☐ El nombre contiene entre 3 y 100 caracteres.
- ☐ El precio es obligatorio y mayor que cero.
- ☐ El precio admite como máximo dos cifras decimales.
- ☐ Los importes monetarios se representan en Java mediante BigDecimal.
- ☐ El stock inicial es un número entero mayor o igual que cero.
- ☐ Antes de insertar un producto, se comprueba que su categoría existe en MySQL.
- ☐ Antes de insertar un producto, se comprueba que su proveedor existe en MySQL.
- ☐ Si alguna de las referencias no existe, el producto se rechaza y se informa del motivo.
- ☐ Los productos válidos se insertan correctamente mediante JDBC.

Consulta y modificación

- ☐ Puedo consultar un producto utilizando su código.
- ☐ La consulta muestra todos los datos relevantes del producto.
- ☐ Si el código no existe, se muestra un mensaje informativo.
- ☐ Puedo modificar el nombre, precio, categoría y proveedor de un producto.
- ☐ Antes de modificarlo, se muestran sus valores actuales.
- ☐ Puedo conservar los valores que no deseo modificar.
- ☐ No puedo cambiar el código identificador del producto.
- ☐ No puedo modificar directamente el stock desde esta opción.
- ☐ Los cambios de categoría o proveedor se validan antes de guardarse.

Eliminación

- ☐ Puedo eliminar un producto que no tenga movimientos de stock asociados.
- ☐ Antes de eliminarlo, se muestran sus datos y se solicita confirmación.
- ☐ Si cancelo la eliminación, el producto permanece en MySQL.
- ☐ No puedo eliminar un producto que tenga movimientos históricos asociados.
- ☐ La aplicación no elimina movimientos para forzar el borrado de un producto.

11.7. Consultas y búsquedas de productos

Desde la opción **7.5. Consultar y buscar productos**, debes comprobar las distintas consultas SQL desarrolladas.

Comprueba lo siguiente:

- ☐ Puedo listar todos los productos almacenados en MySQL.

- ☐ El listado está ordenado por código.
- ☐ Se muestran los nombres de las categorías y proveedores mediante consultas con JOIN.
- ☐ Puedo buscar un producto por su código exacto.
- ☐ Puedo buscar productos introduciendo una parte de su nombre.
- ☐ La búsqueda parcial utiliza LIKE con parámetros preparados.
- ☐ Puedo consultar productos cuyo stock sea inferior a un valor indicado.
- ☐ El umbral de stock solicitado es un número entero mayor o igual que cero.
- ☐ La comparación utiliza stock < umbral, sin incluir productos cuyo stock sea igual al umbral.
- ☐ Los resultados de la consulta de stock se ordenan primero por existencias y después por código.
- ☐ Si no existen coincidencias, la aplicación lo indica claramente.
- ☐ Las consultas utilizan PreparedStatement cuando incorporan datos introducidos por el usuario.
- ☐ Los resultados se recuperan mediante ResultSet.
- ☐ Los datos obtenidos se transforman en objetos Java o DTO cuando corresponde.
- ☐ Las operaciones de consulta no modifican registros.

11.8. Entradas y salidas de stock

Desde la opción **7.6. Registrar entrada o salida de stock**, debes comprobar el funcionamiento del procedimiento almacenado registrar_movimiento.

Comprueba lo siguiente:

- ☐ Puedo registrar una entrada de unidades de un producto existente.
- ☐ Puedo registrar una salida de unidades de un producto existente.
- ☐ La aplicación solicita el código del producto.
- ☐ Antes de registrar el movimiento, se muestra el nombre y el stock actual.
- ☐ La cantidad solicitada debe ser un número entero mayor que cero.
- ☐ Se solicita confirmación antes de ejecutar la operación.
- ☐ La aplicación utiliza CallableStatement para invocar el procedimiento almacenado.
- ☐ No se modifica directamente producto.stock desde Java para registrar estos movimientos.
- ☐ Una entrada correcta incrementa el stock del producto.
- ☐ Una salida correcta disminuye el stock del producto.
- ☐ Cada movimiento correcto genera un registro en movimiento_stock.
- ☐ Después de realizar el movimiento, se muestran el stock anterior y el posterior.
- ☐ No puedo registrar movimientos sobre productos inexistentes.
- ☐ No puedo registrar movimientos con cantidad cero o negativa.
- ☐ No puedo registrar una salida superior al stock disponible.
- ☐ Cuando una operación falla, el stock permanece sin cambios.

- ☐ Cuando una operación falla, no se crea un movimiento en el historial.
- ☐ Si cancelo la operación, no se modifica ningún dato.
- ☐ Las operaciones correctas muestran un mensaje en verde y las rechazadas muestran un error en rojo.

Prueba recomendada: registra una entrada de 5 unidades, comprueba el nuevo stock y consulta el historial. Después, intenta retirar más unidades de las disponibles y verifica que no se produce ningún cambio.

11.9. Historial de movimientos

Desde la opción **7.7. Consultar historial de movimientos**, debes comprobar que los cambios de existencias quedan registrados y pueden consultarse.

Comprueba lo siguiente:

- ☐ Puedo consultar todos los movimientos registrados.
- ☐ Puedo consultar únicamente los movimientos de un producto concreto.
- ☐ Cada movimiento muestra su identificador.
- ☐ Se muestra la fecha y hora del movimiento.
- ☐ Se identifica el producto afectado.
- ☐ Se indica si el movimiento es una ENTRADA o una SALIDA.
- ☐ Se muestra la cantidad de unidades.
- ☐ Se muestran el stock anterior y el stock posterior.
- ☐ Los movimientos aparecen ordenados de forma coherente por fecha e identificador.
- ☐ Si un producto existe pero no tiene movimientos, se muestra un mensaje informativo.
- ☐ Si el producto consultado no existe, la aplicación informa de ello.
- ☐ Las consultas del historial no modifican el stock.
- ☐ Las consultas del historial no generan nuevos movimientos.
- ☐ No se permite eliminar registros del historial desde estas opciones.

11.10. Importación de productos desde memoria a MySQL

Esta funcionalidad constituye uno de los principales puntos de integración entre la UT1 y la UT2.

Desde la opción **7.8. Importar productos de memoria a MySQL**, debes comprobar que la aplicación puede transferir productos desde la colección Java hasta la base de datos.

Comprueba lo siguiente:

- ☐ Puedo importar productos desde un fichero JSON o XML utilizando las funcionalidades de la UT1.
- ☐ Los productos importados quedan disponibles en el inventario en memoria.
- ☐ Al seleccionar la opción 7.8, se muestra el número de productos disponibles para importar.
- ☐ Si el inventario en memoria está vacío, se informa de que no hay productos para transferir.
- ☐ Antes de comenzar la importación, se solicita confirmación.
- ☐ La aplicación recorre los productos almacenados en memoria.
- ☐ Se comprueba que cada producto cumple las validaciones establecidas.

- ☐ Se comprueba que el código del producto no existe ya en MySQL.
- ☐ Se comprueba que la categoría y el proveedor existen en la base de datos.
- ☐ Los productos válidos se insertan correctamente.
- ☐ Los productos duplicados se omiten sin sobrescribir sus datos.
- ☐ Los productos con datos incorrectos o referencias inexistentes se rechazan indicando el motivo.
- ☐ Un producto rechazado por una validación previa no impide importar los demás productos válidos.
- ☐ La aplicación no crea automáticamente categorías o proveedores inexistentes.
- ☐ Las inserciones aceptadas se realizan mediante PreparedStatement.
- ☐ El lote de inserciones se ejecuta dentro de una transacción JDBC.
- ☐ Si las inserciones se completan correctamente, se ejecuta commit().
- ☐ Si se produce un error SQL inesperado durante el lote, se ejecuta rollback().
- ☐ Después de un rollback(), no quedan confirmadas inserciones parciales del lote.
- ☐ Al finalizar, se muestra un resumen con el número de productos insertados, omitidos y rechazados.
- ☐ Los productos permanecen en el inventario en memoria después de importarlos a MySQL.
- ☐ La importación no activa una sincronización automática entre memoria y base de datos.
- ☐ Si repito la importación, los productos existentes no se duplican.

Prueba recomendada: reproduce el ejemplo guiado del apartado 6.1 utilizando tres productos: dos válidos y uno con un proveedor inexistente. Comprueba el resultado, da de alta el proveedor que falta y repite la importación.

11.11. Gestión de transacciones JDBC

Desde la opción **7.9. Demostración de transacción con commit/rollback**, debes demostrar que sabes controlar una transacción directamente desde Java.

Comprueba lo siguiente:

Escenario A. Confirmación de una transacción

- ☐ He seleccionado identificadores de categorías de prueba que no están siendo utilizados.
- ☐ He desactivado la confirmación automática mediante setAutoCommit(false).
- ☐ He insertado dos categorías nuevas mediante PreparedStatement.
- ☐ He ejecutado commit() después de completar correctamente las inserciones.
- ☐ He comprobado mediante una consulta SQL que ambas categorías existen en MySQL.
- ☐ La aplicación muestra un mensaje de confirmación en verde.

Escenario B. Cancelación de una transacción

- ☐ He iniciado una nueva transacción JDBC.
- ☐ He insertado una categoría de prueba.
- ☐ He provocado intencionadamente un error de clave duplicada en una segunda inserción.
- ☐ He capturado la excepción SQLException.
- ☐ He ejecutado rollback() para cancelar la transacción.

- ☐ He comprobado que la primera categoría del intento fallido tampoco se ha almacenado.
- ☐ La aplicación informa de que se ha producido el error esperado y de que la transacción se ha revertido.

Comprobaciones adicionales

- ☐ He restaurado el estado de autoCommit al finalizar.
- ☐ He cerrado correctamente los recursos JDBC utilizados.
- ☐ He documentado los identificadores empleados en las pruebas.
- ☐ La demostración no altera ni elimina datos reales del inventario.
- ☐ He registrado evidencias que permiten comprobar tanto el funcionamiento de commit() como el de rollback().

11.12. Actividad comparativa con SQLite

La actividad SQLite es obligatoria y se desarrolla dentro del mismo proyecto Maven, mediante una clase ejecutable independiente de los menús principales de DataStock.

Comprueba lo siguiente:

- ☐ He incorporado al proyecto Maven la dependencia JDBC necesaria para utilizar SQLite.
- ☐ He creado una clase ejecutable independiente para la demostración, por ejemplo, DemoSQLite.
- ☐ La demostración puede ejecutarse sin necesidad de iniciar el menú principal de DataStock.
- ☐ He creado o abierto correctamente una base de datos SQLite almacenada en un fichero local.
- ☐ He creado la tabla producto_demo con la estructura indicada en el apartado 7.
- ☐ He utilizado JDBC para establecer la conexión.
- ☐ He insertado los registros de prueba solicitados.
- ☐ He realizado una consulta para recuperar los productos almacenados.
- ☐ He mostrado los resultados de la consulta por consola.
- ☐ He comprobado qué sucede cuando vuelvo a ejecutar la demostración.
- ☐ La ejecución repetida no provoca duplicados no controlados.
- ☐ He cerrado correctamente la conexión y los demás recursos JDBC.
- ☐ He documentado en el README las principales diferencias entre MySQL y SQLite.
- ☐ He explicado qué significa que SQLite sea un gestor embebido y MySQL un gestor independiente.
- ☐ He registrado los casos de prueba CP-SQLITE-01, CP-SQLITE-02 y CP-SQLITE-03.

11.13. Buenas prácticas de programación y seguridad

Además de implementar las funcionalidades, debes comprobar que tu código es legible, está organizado y utiliza correctamente JDBC.

Comprueba lo siguiente:

- ☐ Las clases, métodos y variables tienen nombres descriptivos.
- ☐ Se respetan las convenciones de nomenclatura de Java.
- ☐ Las constantes se declaran y utilizan correctamente.
- ☐ El código está organizado en clases y métodos con responsabilidades claras.

- ☐ No he concentrado todas las operaciones JDBC en un único método excesivamente largo.
- ☐ Evito repetir innecesariamente el mismo código.
- ☐ Utilizo PreparedStatement para las operaciones que incorporan valores proporcionados por el usuario.
- ☐ No construyo sentencias SQL concatenando directamente los datos introducidos por consola.
- ☐ Utilizo CallableStatement para ejecutar el procedimiento almacenado.
- ☐ Utilizo ResultSet para recuperar y procesar los resultados de las consultas.
- ☐ Utilizo BigDecimal para representar importes monetarios.
- ☐ Gestiono adecuadamente las excepciones SQLException.
- ☐ Utilizo try-with-resources para cerrar conexiones, sentencias y resultados cuando corresponde.
- ☐ Los mensajes de error son comprensibles para el usuario.
- ☐ No muestro contraseñas ni información sensible en la consola.
- ☐ No he publicado credenciales en GitHub.
- ☐ Los métodos y clases relevantes están documentados mediante comentarios o Javadoc.
- ☐ El proyecto se puede compilar desde Maven sin depender de configuraciones exclusivas de mi IDE.

11.14. Pruebas funcionales y tratamiento de errores

No basta con comprobar que las operaciones funcionan cuando el usuario introduce datos correctos. También debes verificar qué sucede ante situaciones incorrectas.

Comprueba lo siguiente:

- ☐ He ejecutado los casos de prueba JDBC indicados en el apartado 6.
- ☐ He registrado los resultados de las pruebas en la plantilla facilitada.
- ☐ He probado operaciones correctas de inserción, consulta, modificación y eliminación.
- ☐ He probado identificadores duplicados.
- ☐ He probado códigos de producto con caracteres no permitidos.
- ☐ He probado nombres vacíos o con longitudes incorrectas.
- ☐ He probado precios negativos, iguales a cero o con demasiados decimales.
- ☐ He probado valores de stock negativos.
- ☐ He probado productos con categorías o proveedores inexistentes.
- ☐ He probado eliminaciones bloqueadas por claves foráneas.
- ☐ He probado consultas que no devuelven resultados.
- ☐ He probado movimientos de stock válidos.
- ☐ He probado una salida con cantidad superior a las existencias.
- ☐ He comprobado que una operación fallida no modifica indebidamente los datos.
- ☐ He probado la importación de productos duplicados desde memoria.
- ☐ He comprobado el funcionamiento de commit() y rollback().

- ☐ He comprobado el comportamiento de la aplicación cuando MySQL no está disponible.
- ☐ He verificado que los errores se muestran en rojo y los éxitos en verde.
- ☐ He comprobado que la aplicación continúa funcionando después de un error controlado.

Importante: cada prueba registrada debe incluir, como mínimo, el identificador del caso, la operación realizada, los datos utilizados, el resultado esperado, el resultado obtenido y la indicación de si la prueba ha sido superada.

11.15. Comprobación de la persistencia de los datos

Una de las diferencias fundamentales entre la UT1 y la UT2 es que MySQL permite conservar los datos aunque la aplicación deje de ejecutarse.

Comprueba lo siguiente:

- ☐ He insertado categorías, proveedores y productos en MySQL.
- ☐ He cerrado completamente DataStock.
- ☐ He vuelto a ejecutar la aplicación.
- ☐ He comprobado que el inventario en memoria comienza vacío si no se realiza ninguna carga.
- ☐ He accedido al menú MySQL y consultado los productos almacenados.
- ☐ Los productos siguen disponibles después de reiniciar la aplicación.
- ☐ Las categorías y proveedores también permanecen almacenados.
- ☐ Los movimientos de stock registrados siguen apareciendo en el historial.
- ☐ Las consultas JDBC recuperan los datos directamente desde MySQL.
- ☐ He comprobado que no es necesario volver a importar los ficheros JSON o XML para consultar los productos que ya están almacenados en la base de datos.

11.16. Documentación, repositorio y entrega

Antes de entregar, debes comprobar que el repositorio contiene todo lo necesario para revisar y ejecutar tu trabajo.

Comprueba lo siguiente:

- ☐ El código fuente de la UT2 está incorporado al mismo repositorio GitHub utilizado en la UT1.
- ☐ El proyecto contiene un fichero pom.xml válido.
- ☐ Las dependencias Maven están correctamente declaradas.
- ☐ El repositorio conserva las funcionalidades de la UT1 y añade las de la UT2.
- ☐ El código fuente está organizado en paquetes y clases comprensibles.
- ☐ El repositorio incluye el script SQL proporcionado o las instrucciones necesarias para localizarlo y ejecutarlo.
- ☐ El README explica cómo preparar la base de datos MySQL.
- ☐ El README explica cómo configurar la conexión JDBC sin publicar contraseñas.
- ☐ El README indica cómo compilar y ejecutar DataStock.
- ☐ El README explica cómo ejecutar la demostración independiente de SQLite.
- ☐ El README incluye la comparación solicitada entre MySQL y SQLite.
- ☐ He completado el registro de pruebas con los casos correspondientes a la UT2.

- ☐ He incluido evidencias de las operaciones principales y de las situaciones de error.
- ☐ He comprobado que las capturas o evidencias permiten identificar claramente la operación realizada y su resultado.
- ☐ He realizado commits descriptivos que reflejan el desarrollo de la práctica.
- ☐ He creado la etiqueta Git correspondiente a esta segunda entrega, siguiendo las instrucciones establecidas.
- ☐ He comprobado que todos los cambios están confirmados y enviados al repositorio remoto.
- ☐ He verificado que el repositorio no contiene contraseñas ni otros datos sensibles.
- ☐ He revisado que la versión entregada compila y se ejecuta correctamente.
- ☐ He entregado en Moodle los enlaces y documentos solicitados.

11.17. Prueba final de integración

Como última comprobación, deberás ejecutar un recorrido completo de DataStock que permita demostrar que las funcionalidades de la UT1 y la UT2 trabajan conjuntamente.

Para ello, reproduce el ejemplo guiado del apartado 6.1 y verifica que puedes:

1. Importar varios productos desde un fichero JSON o XML.
2. Consultar los productos almacenados en memoria.
3. Establecer correctamente la conexión con MySQL.
4. Dar de alta las categorías y proveedores necesarios.
5. Importar los productos válidos desde memoria a MySQL.
6. Detectar y rechazar un producto cuyo proveedor no existe.
7. Consultar los productos que se han almacenado correctamente.
8. Cerrar DataStock y volver a ejecutarlo.
9. Comprobar que los productos siguen almacenados en MySQL.
10. Dar de alta el proveedor que faltaba.
11. Repetir la importación y comprobar que se inserta el producto pendiente sin duplicar los existentes.
12. Registrar una entrada de stock.
13. Registrar una salida de stock.
14. Consultar el historial y verificar que ambos movimientos están registrados.
15. Intentar una salida superior al stock disponible y comprobar que la operación se rechaza.
16. Verificar que el stock y el historial no se han modificado después de la operación fallida.

La prueba de integración se considerará superada cuando todas estas operaciones puedan realizarse desde los menús de DataStock, sin modificar manualmente los registros de MySQL para conseguir los resultados esperados.

Antes de entregar

Si alguna de las comprobaciones anteriores no se cumple, revisa el requisito funcional correspondiente, corrige el problema y repite la prueba.

Recuerda que **la evaluación no se limita a comprobar que existen las clases o los métodos solicitados**. También se valorará que las funcionalidades estén correctamente integradas, que los datos mantengan su integridad, que los errores se gestionen adecuadamente y que puedas demostrar el funcionamiento de tu aplicación.

Tu objetivo es entregar una segunda versión de DataStock funcional, documentada y verificable, que incorpore persistencia mediante JDBC y MySQL sin perder las funcionalidades desarrolladas en la UT1.